

项目复盘：从静态地图到交互式动物园图册

1.初始目标与思路

最初的目标是为儿子打造一个专属的“交互式动物园图册”，解决纸质地图信息有限且无法互动的痛点。核心思路是将静态图片转化为动态网页，但考虑到地图上信息密集，整图 OCR 识别效果差且难以对应具体区域，手工录入又过于繁琐。因此，我设计了“半自动标注”的方案：利用本地部署的视觉大模型（VLM），通过“手动画框+小模型 OCR”的方式，实现区域划分与内容识别的同步进行，既保证了数据的准确性，又大幅降低了录入成本。

2.任务拆解与迭代过程

项目经历了从规划到落地的多轮迭代，主要分为三个阶段：
架构规划与数据层：前 4 轮对话专注于方案对齐，确立了“管理端+用户端”分离的架构。设计了通用的 JSON 数据模型，支持多类型点位（动物、设施等），并决定使用 IndexedDB 进行本地存储。

核心功能开发：开发了画框工具与 LLM 适配层。在识别环节，从最初的单一 Ollama 接口扩展到支持配置化调用（Ollama/llama.cpp/OpenAI），并增加了超时重试机制以应对本地模型推理不稳定的问题。

交互体验优化：针对地图浏览体验，增加了缩放与平移功能（panzoom），并修复了缩放后坐标偏移的 Bug。后期根

据反馈，增加了“点击地图区域直接编辑”的快捷操作，以及将动物个体名字（如“圆圆”）作为独立字段展示，提升了图册的丰富度。

3.大模型的表现评估

表现优异处：在代码生成与架构设计上表现出色。它能迅速理解“本地化部署”的需求，生成了无后端依赖的纯前端架构（HTML+JS+IndexedDB）。对于复杂的逻辑（如 SVG 坐标映射、异步 OCR 队列管理），它能提供高质量的代码实现。

跑偏与不足：在处理视觉识别任务时，对“卡通风格”地图的识别能力预判过于乐观，导致初期识别准确率波动，需要人工校对。此外，在涉及浏览器底层机制（如 CORS 跨域、File 协议限制）时，偶尔会忽略环境配置的提示，导致用户端报错。

4.问题与修复策略

CORS 跨域问题：浏览器无法直接请求本地 Ollama 服务。

策略：通过设置环境变量 `OLLAMA_ORIGINS="*"` 并重启服务解决，同时在工具中增加了错误提示引导。

缩放坐标偏移：地图缩放后，绘制的矩形框坐标与实际位置不重合。**策略：**利用 SVG 的 `getScreenCTM` 方法自动适配缩放矩阵，确保坐标计算的准确性。

OCR 识别卡顿：本地模型推理慢导致请求挂起。**策略：**在

ocr.js 中引入 AbortController 实现超时切断，并增加指数退避的自动重试机制，配合实时日志面板，让用户清晰看到识别进度与错误原因。

数据丢失风险：IndexedDB 数据仅存在浏览器缓存中。策略：开发“导出/导入JSON”功能，将底图（Base64）与标注数据打包下载，实现数据的持久化备份。

5. 最终总结

通过这次实践，我对“算法思维”的理解从单纯的代码逻辑转向了“人机协作的系统工程”。

容错设计至关重要：本地大模型并非 100% 可靠，系统设计必须包含“AI 预处理+人工校对”的闭环，不能盲目信任自动化。

数据与视图分离：将地图数据抽象为通用的 JSON 结构，使得这个工具不仅能做动物园地图，未来也能轻松扩展为校园导览或博物馆图册。

本地化的价值：利用本地算力处理隐私数据和特定任务，配合 Web 技术的灵活性，是个人开发者构建实用工具的极佳路径。

6. 作品的真实价值

这个项目不仅是一个技术练习，更是送给儿子的真实礼物。后续我计划继续添加“动物明星日记”和“打卡成就系统”，让它真正成为孩子探索自然的数字伴侣。下一步，我希望能

加入语音合成（TTS）功能，让地图能“开口说话”，最后加上“测测你是动物园几级学者”互动小游戏。